Hardware-Efficient Attention: A Survey and GPU Acceleration Study

Qi Shuqi, Jiang Yiwei, Lin Zheyu, Zhang Jianming

Tsinghua University

Abstract

In modern transformer models, the attention mechanism remains the only component with quadratic time complexity $O(N^2)$, while all other operations scale linearly with sequence length N. As generative applications increasingly demand long-context processing, improving the computational efficiency of attention has become critical. This paper focuses specifically on hardware-efficient attention methods, which optimize attention computation by leveraging hardware characteristics and GPU-specific acceleration techniques. We provide an in-depth review of state-of-the-art approaches including Flash Attention, Sage Attention, and Sparge Attention, along with key GPU acceleration strategies such as low-precision arithmetic, operator fusion, recomputation, and tiling. Furthermore, we present experimental results from our implementation of a transformer model integrated with a custom Triton kernel for Flash Attention 2, alongside performance profiling and benchmarking. We also propose innovations applied to the Sage3 attention mechanism. Our work offers both a systematic analysis of hardware-oriented attention optimization and practical insights for achieving high-performance inference and training on GPU platforms.

1 Introduction

Attention serves as the core computational mechanism in transformer models(Vaswani et al. 2017), driving their remarkable success across domains such as natural language processing, computer vision, and multimodal generation. However, a fundamental bottleneck limits its broader application: the quadratic time and memory complexity $O(N^2)$ of standard self-attention with respect to sequence length N. As generative and understanding tasks increasingly demand long-context capabilities—from extended document processing to high-resolution video generation—this computational overhead becomes prohibitive, especially under strict latency or hardware constraints.

In response, significant research efforts have been directed toward developing efficient attention methods that reduce computational cost while preserving representational capacity. These approaches can be broadly categorized into several classes, including sparse attention, linear attention, and compact attention. Among them, hardware-efficient attention has emerged as a highly practical direction that accelerates the original attention mechanism without altering its mathematical formulation. By leveraging low-level hardware features and GPU-specific optimizations, these methods achieve substantial speedup and memory reduction on modern accelerators.

While existing surveys often cover efficient attention from an algorithmic perspective, this paper focuses specifically on hardware-efficient attention and its system-level implementations. We provide a systematic review of key techniques including FlashAttention(Dao et al. 2022), SageAttention(Zhang et al. 2024b), and SpargeAttention(Zhang et al. 2025b), along with essential GPU acceleration strategies such as operator fusion, tensor core utilization, and blocked computation. Furthermore, we present experimental evaluations and custom implementations—including a Triton kernel for FlashAttention-2(Dao 2023) and innovations applied to Sage3(Zhang et al. 2025a)—offering practical insights into real-world performance optimization.

The remainder of this paper is organized as follows: Section 2 introduces background on standard attention and GPU architecture. Section 3 details prominent hardware-efficient attention methods. Section 4 discusses GPU acceleration techniques supporting these approaches. Section 5 presents experimental results and profiling analyses. Finally, Section 6 concludes the paper and suggests future directions.

2 Background

2.1 Standard Attention

Given input sequences $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N\times d}$, where N is the sequence length and d is the head dimension, the standard attention mechanism computes the output $\mathbf{O} \in \mathbb{R}^{N\times d}$ through the following steps:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N\times N},$$
$$\mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N\times N},$$
$$\mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N\times d},$$

where the $\mathrm{softmax}$ is applied row-wise. For multi-head attention (MHA), this computation is performed in parallel across multiple heads and batch dimensions.

The backward pass proceeds as follows. Let $d\mathbf{O} \in \mathbb{R}^{N\times d}$ be the gradient of the loss with respect to $\mathbf{O}$. Then:

Copyright © 2026, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

$$
\begin{aligned}
d\mathbf{V} &= \mathbf{P}^\top d\mathbf{O}, \\
d\mathbf{P} &= d\mathbf{O}\mathbf{V}^\top, \\
d\mathbf{S} &= d\mathrm{softmax}(d\mathbf{P}), \\
d\mathbf{Q} &= d\mathbf{S} \cdot \mathbf{K}, \\
d\mathbf{K} &= \mathbf{Q} d\mathbf{S}^\top,
\end{aligned}
$$

where $d\mathrm{softmax}$ denotes the row-wise gradient of the softmax operation. Specifically, for a vector $\mathbf{s}$ with softmax output $\mathbf{p} = \mathrm{softmax}(\mathbf{s})$ and upstream gradient $d\mathbf{p}$, the gradient $d\mathbf{s}$ is given by:

$$
d\mathbf{s} = \left(\mathrm{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^\top\right) d\mathbf{p}.
$$

Standard attention implementations materialize the matrices $\mathbf{S}$ and $\mathbf{P}$ in high-bandwidth memory, requiring $O(N^2)$ memory. Since $N \gg d$ in typical scenarios (e.g., $N \in [10^3, 10^4]$, $d \in [64, 128]$), and most operations are memory-bandwidth bounded, the large number of memory accesses results in slow wall-clock execution. Additionally, $\mathbf{P}$ must be stored during the forward pass for use in the backward pass, further increasing memory requirements.

2.2 GPU

Modern GPUs are massively parallel processors with a multi-tiered memory architecture. Using NVIDIA's CUDA framework as a reference, the computational power of GPUs stems from multiple Streaming Multiprocessors (SMs). Each SM contains numerous CUDA cores for general computation and specialized tensor cores for accelerated matrix multiplication. The collective performance of these cores defines the GPU's computational throughput, denoted as C_t (operations per second), commonly measured in FLOPS.

To feed data to these compute units, GPUs employ a memory hierarchy. Each SM has a fast, on-chip shared memory, while all SMs share a larger but slower global memory (typically High Bandwidth Memory (HBM)), with bandwidth B_w (bytes per second).

Consider a matrix multiplication $C = AB$, with $A \in \mathbb{R}^{M \times K}$, $B \in \mathbb{R}^{K \times N}$, and $C \in \mathbb{R}^{M \times N}$, all stored as floats. The matrices are moved from global to shared memory for computation, and results are written back. The total I/O volume is $4(MK + KN + MN)$ bytes, and the total computation is $2MKN$ operations. The I/O time is $T_{\text{I/O}} = 4(MK + KN + MN)/B_w$, and the computation time is $T_{\text{compute}} = 2MKN/C_t$. Due to pipelining that overlaps computation and I/O, the overall latency is $\max(T_{\text{I/O}}, T_{\text{compute}})$. The operation is memory-bound if $T_{\text{I/O}}$ dominates, and compute-bound if T_{compute} dominates.

Designing efficient GPU operations requires careful consideration of the memory hierarchy. A standard procedure involves loading data from global to shared memory, performing computations, and writing results back. Through pipelining, computation and memory I/O can be overlapped, making the overall execution time dependent on the slower of the two processes. If computation time exceeds I/O time, the operation is compute-bound; conversely, if I/O time dominates, it is memory-bound.

The naive implementation of attention is memory-bound due to the materialization of the large intermediate matrices $S = QK^\top$ and $P = \mathrm{softmax}(S)$, each of size $\mathcal{O}(N^2)$. Storing and accessing these matrices in global memory incurs substantial I/O overhead. For example, on an NVIDIA A100 GPU with a global memory bandwidth of 2.0 TB/s and a peak FP16 throughput of 312 TFLOPS, consider input matrices Q, K, V of size $[100K, 64]$. The intermediate matrices S and P would each be of size $[100K, 100K]$, requiring approximately 19 GB each in half-precision. The total FLOPs for the two matrix multiplications is $4N^2d$. The computation time is: $T_{\text{compute}} = \frac{2\times(2N^2d)}{C_t} = \frac{4\times 1e10\times 64}{312\times 1e12} \approx 8.2\,\text{ms}$, The I/O time, dominated by writing and reading S and P, is: $T_{I/O} = \frac{2\times(\text{size of } S + \text{size of } P)}{B_w} = \frac{8\times 1e10\,\text{bytes}}{2\times 1e12\,\text{bytes/s}} \approx 40\,\text{ms}$,

Here, the I/O time is nearly five times longer than the computation time, confirming that naive attention is severely limited by memory bandwidth.

3 Hardware-Efficient Attention

3.1 Flash Attention

To speed up attention on hardware accelerators such as GPU, Dao proposes an algorithm to reduce the memory reads/writes while maintaining the same output (without approximation).

A large matrix multiplication can be divided into small blocks using $Tiling$ technology, and then the final correct result can be obtained by summing the results of each block matrix multiplication.

Regrettably, the Softmax calculation in Attention does not have this additive property; it relies on a global denominator. The core issue that Flash Attention and online softmax aim to solve is how to decouple the algorithm itself from this global dependency, thereby enabling the use of Tiling for rapid on-chip computation. Judging from the results, compared to the original $3 - pass$ algorithm, online-softmax is a $2 - pass$ algorithm, while Flash Attention is a $1 - pass$ algorithm.

Online Softmax Construct the sequence d_i defined as follows:

$$
d_i = \sum_{j=1}^{i} e^{x_j - m_i}.
$$

The key property is that for all $i \in [1, N]$, when $i = N$:

$$
d_N = d'_N.
$$

So relation between d_i and d_{i-1} is:

$$
d_i = d_{i-1}\, e^{m_{i-1} - m_i} + e^{x_i - m_i}.
$$

Now the recursion between d_i and d_{i-1} depends only on m_{i-1} and m_i. Hence we can compute both d_i and m_i in the same loop; when the loop reaches $i = N$:

$$
d_N = d'_N.
$$

Since $onlinesoftmax$ no longer needs to store the intermediate matrix X, GPU memory is saved; however, the number of arithmetic operations remains unchanged and. In

Algorithm 1: $2-pass$ online softmax

for $i \leftarrow 1, N$ **do**
  $m_i \leftarrow \max(m_{i-1}, x_i)$
  $d'_i \leftarrow d'_{i-1} e^{m_{i-1}-m_i} + e^{x_i - m_i}$
end for
for $i \leftarrow 1, N$ **do**
  $a_i \leftarrow \frac{e^{x_i - m_N}}{d'_N}$
end for

fact, the accesses to high-bandwidth memory (HBM) increase because Q and K must be loaded repeatedly.(Dao et al. 2022) Compared with the $3-pass$ baseline, the $2-pass$ algorithm reduces one full global load of (Q, K) and eliminates one on-the-fly recomputation of $\exp(\cdot)$, because in the first pass of the two-pass scheme the x_i are reused by the two subsequent computations.

Flash Attention V1 Although there is no $1-pass$ softmax, O can be determined in $1-pass$. That is Flash Attention.

Algorithm 2: Flash Attention

for $i \leftarrow 1, N$ **do**
  $C_i \leftarrow Q[k,:] \cdot K^\top[:,i]$
  $m_i \leftarrow \max(m_{i-1}, C_i)$
  $d_i \leftarrow d_{i-1} \cdot e^{m_{i-1}-m_i} + e^{C_i - m_i}$
  $O[k,:] \leftarrow O[k,:] + e^{C_i - m_i} d_i V[i,:]$
end for

The core contribution of Flash Attention is precisely to break the global dependency of softmax into tile-level recursions, thereby enabling efficient $Tiling$ of attention for the first time. Standard attention must first obtain the entire row of scores and cannot compute and normalize tile-by-tile like Flash Attention.In tiling scheme, the K matrix is split into multiple tiles (the same can be done for Q).Each small tile is loaded into $SRAM$, the local products are computed, and the remaining steps follow.Algorithmically, Q, K, and V now need to be loaded only once, allowing the entire attention computation to be completed inside the kernel.

In backward pass, using Tiling, we split Q,K , and V into tiles, load each tile into $SRAM$, and then compute the current tile's S and P values via online re-computation.(Dao 2023)

Flash Attention V2/V3 Flash Attention V1 can be inefficient due to suboptimal work partitioning between different thread blocks and warps on the GPU, causing either low-occupancy or unnecessary shared memory reads/writes. Flash Attention V2 usess better work partitioning to address these issues. It tweaks the algorithm to reduce the number of non-matmul FLOPs, parallelizes the attention computation, even for a single head, across different thread blocks to increase occupancy, and within each thread block, distributes the work between warps to reduce communication through shared memory. (Dao 2023)

Flash Attention V3 introduced the producer-consumer asynchronous mechanism which which further enhances parallelism. It also supports FP8 (Float8) low-precision hardware acceleration which which enhances throughput and reduces precision loss.(Shah et al. 2024) These improvements are particularly beneficial for processing large Transformer models and long sequences, significantly enhancing the training and inference speed of the model.

3.2 SageAttention

The core idea of SageAttention can be summarized as a "quantized version of FlashAttention": it systematically introduces low-bit quantization strategies on top of the FlashAttention framework to alleviate the substantial storage, transmission, and computation overhead of attention mechanisms (Zhang et al. 2024b). By compressing key matrices such as Q, K, V, and P into low-precision numerical formats, SageAttention significantly reduces memory bandwidth consumption and improves compute utilization while maintaining model accuracy.

From the algorithmic perspective, attention computation essentially involves $O(N^2d)$ matrix multiplications. At the same time, the overhead of quantization and dequantization is mainly proportional to the matrix size (about $O(N^2)$ or $O(Nd)$). Therefore, if quantization can fully leverage hardware characteristics, it is possible to achieve significant speedups with virtually no additional complexity.

Efficiency–Accuracy Trade-off The key challenge faced by the SageAttention family lies in balancing efficiency and accuracy across diverse hardware architectures (Zhang et al. 2024b,a, 2025c,a). On one hand, efficiency demands that quantization strategies be closely tailored to specific hardware capabilities, as different architectures (e.g., NVIDIA Ada versus Blackwell) exhibit varying levels of support for low-bit compute units. On the other hand, maintaining accuracy is complicated by the inevitable weakening of numerical representation power under low-bit quantization and the amplification of outlier effects, which necessitates carefully designed quantization and accumulation mechanisms to mitigate error propagation.

Intuitively, the fewer bits used, the greater the potential speedup; however, this also results in a larger quantization error. The evolution of SageAttention reflects this "bit-by-bit" exploration: starting from Sage1 with INT8, to Sage2/2++ with INT4, and then to Sage3 with FP4, to approach the optimal numeric representation supported by different GPU architectures (e.g., FP4 Tensor Cores in Blackwell GPUs). At the same time, SageAttention goes beyond bit-width reduction by introducing innovations such as *numerical smoothing*, *two-level accumulation*, and *microscaling*, addressing the "non-uniformity" of low-bit representations.

Technical Evolution. SageAttention's development can be divided into four major versions, as summarized in Table 1(FA2=FlashAttention2)(Zhang et al. 2024b).

As shown in Table 1:

Sage1: Proposed block-wise INT8 quantization for Q and K, while keeping P, V in FP16 and using FP16 accumulators

Table 1: Comparison of SageAttention Versions: Core Improvements, Challenges, and Innovations

Version	**Core Improvements**	**Main Challenges**	**Solution Approaches**	**Performance Gains**	**Key Innovations**
Sage1	• Q,K $\rightarrow$ INT8 (block quant.) • P,V $\rightarrow$ FP16 w/ FP16 accumulator	• Channel-wise outliers in K • Direct P,V $\rightarrow$ INT8 $\Rightarrow$ accuracy loss	• **Smooth K** (subtract channel mean) • FP16 accumulator boosts PV comp.	**2.1×** faster than FA2	• First systematic attention quantization • Solves channel-wise outliers
Sage2	• Q,K $\rightarrow$ INT4 (per-thread) • P,V $\rightarrow$ FP8	• INT4 range too narrow • FP8 accum. only 22-bit	• **Q+K smoothing** • Per-thread quant. (indep. scale) • Two-level accumulation • (Opt.) smooth V	**3×** faster than FA2	• Per-thread quantization • Two-level accumulation
Sage2++	• Q,K $\rightarrow$ INT4/INT8 • P,V $\rightarrow$ FP8 • FP16 accum. Matmul	• FP16 accum. limited range • Prone to overflow	• Constrain quant. range • Delayed FP32 buffering ($\downarrow$type conv.)	**3.9×** faster than FA2	• Range constraint • Delayed buffering
Sage3	• FP4 Tensor Core (Blackwell) • Microscaling quant. (1×16) • Explores 8-bit training	• FP4 has only 15 values • P $\in [0, 1]$ decimals, scale insufficient • Gradient error accum. severe	• FP4 microscaling (1×16) • Two-level quant. of P • dP preserved in FP16	**5×** faster than FA2	• Two-level P quantization • Gradient precision preservation

to stabilize computation. Introduced K smoothing to handle channel-wise outliers. Achieved $\sim$2.1$\times$ speedup over FlashAttention2 with minimal accuracy degradation.

Sage2: Pushed Q, K down to INT4 with per-thread quantization and represented P, V in FP8. To mitigate precision loss, introduced two-level accumulation for FP8 Matmul, combined with Q+K smoothing. Achieved $\sim$3$\times$ speedup over FlashAttention2.

Sage2++: Focused on engineering refinements. Used FP16 accumulators with `mma.f16.f8.f8.f16`, constrained product ranges ($P_r \times V_r \leq 2047$), and delayed FP32 buffering to avoid overflows and type-casting overhead. Achieved $\sim$3.9$\times$ speedup over FlashAttention2, maintaining Sage2's accuracy.

Sage3: Leveraged Blackwell-native FP4 Tensor Cores with microscaling (1$\times$16) quantization, and pioneered 8-bit training with gradient preservation in FP16. On RTX5090, achieved 1038 TOPS compute throughput, $\sim$5$\times$ faster than FlashAttention2. Fine-tuning with 8-bit attention preserved accuracy, though pretraining remained unstable due to convergence challenges.

Comparison with FlashAttention: While FlashAttention improves memory access and parallelism at higher precision (FP16/FP8), SageAttention emphasizes numeric compression and hardware co-design. In practice, SageAttention achieves 2–5$\times$ speedups over FlashAttention2 across Ada and Blackwell GPUs, with negligible accuracy loss, making the two approaches complementary.

Strengths SageAttention demonstrates strong generality and plug-and-play compatibility, as it can be applied without modifying the underlying model architecture—unlike spatiotemporal sparsification methods, which often require structural adjustments. Furthermore, it achieves high computational speed while maintaining competitive accuracy, offering substantial acceleration with minimal degradation in model performance.

Limitations Several limitations warrant consideration. First, current activation smoothing techniques remain largely empirical and may lack generalizability across diverse tasks and architectures. Second, the method exhibits strong hardware dependence, particularly on GPU support for low-bit arithmetic units, raising concerns regarding longevity across hardware generations. Finally, while fine-tuning with 8-bit attention proves effective, full pretraining remains unstable due to gradient variances that frequently exceed the representational range of FP8 precision.

Observations Beyond Hardware. Beyond hardware acceleration, SageAttention reveals an intriguing empirical observation: The importance of Q and K channels is not uniform. As illustrated in Figure 1, only a subset of channels contributes significantly to attention scores, while many channels play a minor role. This suggests that attention mechanisms may exhibit an inherent redundancy across embedding dimensions.

This finding is not directly tied to hardware acceleration but offers potential theoretical and algorithmic implications. For instance, one could explore *dimension reduction* of Q and K (e.g., from d to d_{small}), focusing computation on the most informative channels. Such a strategy could simultaneously reduce computational complexity and potentially enhance the contextual expressiveness of attention.

Intuitively, this hypothesis aligns with linguistic properties: the strength of word–context associations is often driven more by *syntactic roles* (e.g., part-of-speech) than by full semantic content. This perspective motivates future directions such as adaptive channel pruning, dynamic quantization granularity, or selective scaling, which could further optimize attention beyond its hardware execution path.

3.3 SpargeAttention

Sparse attention techniques aim to mitigate this by computing attention only for a select subset of "important" token pairs, reducing complexity to $O(N)$ or $O(N \log N)$. However, existing methods often rely on pre-defined patterns (e.g., sliding windows), require model re-training, or only show benefits for extremely long sequences. SpargeAttention, introduced by researchers from Tsinghua University, breaks these limitations by offering a training-free, plug-and-play sparse attention solution that delivers substantial speedups even on moderate-length sequences while preserving accuracy.

Previous sparse attention strategies can be categorized as follows:

- **Local Window Sparse**: Tokens only attend to neighbors within a fixed window.
- **Content-Based Sparse**: Tokens dynamically attend to others based on content similarity.
- **Block Sparse**: Attention is computed only between specific blocks of tokens.
- **Global+Local**: Combines local attention with a few global tokens that attend to all.

These methods often face common issues: sparse patterns rely on priors, risk missing critical dependencies, require complex implementation, and frequently necessitate model retraining.

Core Innovations of SpargeAttention SpargeAttention is designed as a training-free, general-purpose, and accuracy-preserving sparse attention solution. Its core objective is to accelerate inference across diverse models—including language, vision, and video architectures—for moderate context lengths ranging from 4K to 32K tokens.(Zhang et al. 2025b)

SpargeAttention offers several key advantages: it is training-free and can be applied directly to pre-trained models without fine-tuning; it demonstrates strong multi-modal generalizability across language, video, and image models; and it maintains end-to-end task accuracy while achieving 4–7× inference speedup over FlashAttention. The method is designed as a plug-and-play module, requiring only minimal one-time hyperparameter tuning for deployment.

Technical Challenges and Solutions The framework addresses two major technical challenges. First, it tackles the issue of *generality*: attention sparsity patterns are highly dynamic across different models and layers, and SpargeAttention adapts universally without relying on predefined rules. Second, it ensures *usability* by balancing accurate identification of sparse regions with low computational overhead from the prediction step itself, thereby maintaining both precision and efficiency.

SpargeAttention addresses the aforementioned challenges through a combination of innovative techniques. Its core algorithm can be summarized by the following components:

Compression-Based Sparsity Prediction This is a generic, fast algorithm for predicting sparse regions in the attention matrix P:

- **Selective Compression**: The Query (Q) and Key (K) matrices are selectively compressed to retain critical information while reducing dimensionality.
- **Sparsity Prediction**: The compressed Q and K are used to predict which elements in the attention matrix P will be negligible.
- **TopCdf Operation**: A TopCdf operation is applied to omit the calculations ($QK^\top$ and subsequent PV multiplications) corresponding to the identified sparse regions in P.

Warp-Level Sparse Online Softmax An optimized sparse Online Softmax algorithm operating at the GPU warp level exploits the difference between global and local maximum values calculated during the Online Softmax process. This difference allows the algorithm to safely skip portions of the Value (V) matrix multiplication for elements with negligible attention scores and maximizes hardware utilization and minimizes overhead.

Token Reordering for Vision Models (Optional) For video and image models, SpargeAttention leverages the inherent local similarity of visual tokens: it employs Hilbert reordering to rearrange tokens before attention computation. And the Hilbert space-filling curve preserves spatial locality, grouping similar tokens together. This rearrangement enhances the sparsity pattern, making it more structured and easier to predict accurately.

Integration with Quantization SpargeAttention can be fused with quantization techniques like SageAttention. The combination of sparsity and quantization provides a dual acceleration strategy.

Implementation and Optimization The implementation of SpargeAttention is highly optimized for hardware efficiency, focusing on memory access. To avoid the irregular memory access patterns, many sparse methods:

- Ensure contiguous memory layout for sparse computations.
- Utilize GPU cache hierarchy through prefetching and caching strategies.
- Employ memory coalescing to combine multiple memory transactions.

Overhead Control A critical achievement is the near-elimination of the overhead from the sparsity prediction step itself. Through lightweight algorithms, parallel prediction, and prediction caching, the overhead is reduced to negligible levels across various sequence lengths.

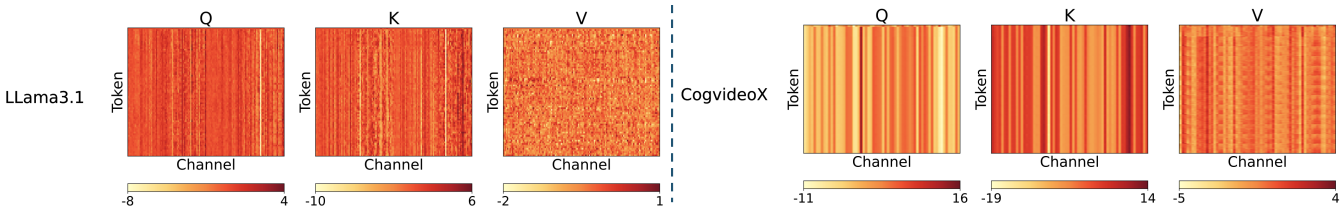

Figure 1: Q/K channel importance heatmap (darker indicates stronger contribution). Adapted from Zhang et al. (2024a).

Table 2: Comparison of Sparse Attention Types

Type	Principle	Advantages	Disadvantages	Examples
Local Window	Each token attends to a fixed local window around it.	Simple to implement, computationally efficient.	May lose important long-range dependencies.	Longformer
Content-Based	Tokens dynamically attend to others based on content similarity.	Dynamic and flexible, content-aware.	Overhead from calculating content similarity.	Various
Block Sparse	Sequence is divided into blocks; attention is computed only between selected blocks.	Balances local and global attention.	Choosing the optimal block size and pattern can be challenging.	BigBird
Global+Local	Combines local attention with a few global tokens that attend to all tokens.	Retains global modeling capacity.	The number and choice of global tokens are heuristic.	BigBird, Longformer

Table 3: Performance of SpargeAttention Across Different Models

Model Type	Task	Sparsity	Speedup	Accuracy
Language Model	Text Generation	60%	4.5×	Lossless
Video Model	Video Generation	55%	4.2×	Lossless
Image Model	Image Generation	65%	4.8×	Lossless
Multimodal Model	Visual Question Answering (VQA)	58%	4.3×	Lossless

End-to-End Acceleration SpargeAttention provides significant overall inference speedups:(Zhang et al. 2025b)

- approximately 2x end-to-end speedup on the Mochi video generation model.
- 1.5-1.8x end-to-end speedup on language models.
- 1.6-1.9x end-to-end speedup on image generation models.

It achieves speeds up to 900 TOPS on an RTX 4090 at 60% sparsity. This represents a 4.5x speedup over FlashAttention-v2 on an A100 (200 TOPS) and enables high-performance inference on consumer-grade GPUs.

Limitations of SpargeAttention

- Requires an initial hyperparameter search for optimal sparsity configuration per model.
- Performance on extremely long sequences (>1M tokens) is less explored.
- Adaptation to highly dynamic sparsity patterns within a single sequence could be improved.

In conclusion, SpargeAttention represents a significant breakthrough in efficient attention computation. By providing a training-free, general-purpose, and accuracy-preserving method, it effectively addresses the quadratic bottleneck of standard attention. Its core innovations—compression-based sparsity prediction, warp-level sparse online softmax, and optional token reordering—enable 4-7x faster inference than FlashAttention while maintaining perfect accuracy across modalities.

This technology holds substantial academic and practical value, facilitating the deployment of large models in resource-constrained and latency-sensitive environments. As sequence lengths continue to grow, training-free sparse attention mechanisms like SpargeAttention are poised to become a standard component for efficient inference in next-generation AI models.

4 Hardware-Oriented Approaches

4.1 Lower Precision Computation

A fundamental approach to GPU acceleration involves the adoption of lower numerical precision, such as transitioning from FP32 to FP16 or even INT8. Such transitions can yield orders-of-magnitude improvements in performance. The underlying principle is straightforward: using fewer bits for numerical representation—including for weights, activations, and gradients—substantially reduces the volume of data that must be moved between memory and computational units. To illustrate, consider a simple element-wise operation like

the ReLU function ($f(x) = \max(0, x)$) applied to a vector of size n:

FP32 Precision

- *Memory access*: One read operation and one conditional write operation (if $x < 0$), with each element occupying 4 bytes
- *Operations*: One comparison and one floating-point operation (FLOP)
- *Arithmetic intensity*: 8 bytes / FLOP

FP16 Precision

- *Memory access*: One read operation and one conditional write operation (if $x < 0$), with each element occupying 2 bytes
- *Operations*: One comparison and one floating-point operation (FLOP)
- *Arithmetic intensity*: 4 bytes / FLOP

Although the number of floating-point operations remains unchanged, the memory bandwidth requirement is halved when using FP16. This effectively doubles available memory bandwidth without altering the computational workload.

However, not all operations are suitable for low-precision computation. For instance, in mixed-precision matrix multiplication, inputs may be stored in 16-bit format to reduce memory footprint and bandwidth pressure, while the core multiplication and accumulation are performed in full FP32 precision. This hybrid approach leverages the benefits of both precision types: reduced data movement through lower storage precision and improved numerical stability through higher-precision intermediate calculations. Modern GPU tensor cores are specifically designed to support such mixed-precision workflows, efficiently computing with FP16 inputs while accumulating partial sums in FP32.

It is important to note that certain operations require higher numerical precision or a larger dynamic range to maintain correctness and stability. Functions such as exponentials, logarithms, or reductions may suffer from overflow, underflow, or significant rounding errors if insufficient precision is used. Therefore, when employing lower-precision arithmetic—especially during training—careful consideration must be given to numerical behavior, scaling strategies, and precision-sensitive operations to ensure model convergence and stability.

4.2 Operator fusion

A critical optimization technique for mitigating memory bandwidth constraints on GPUs is operator fusion. The computational throughput of a GPU is often limited not by its raw processing power, but by the available bandwidth for transferring data from memory to the computational units. A significant, yet often overlooked, source of inefficiency arises from the naive execution of sequences of operations where intermediate results are frequently written to and read from global memory.

In a typical unoptimized execution flow, an input tensor is first loaded from global memory, processed by a single operation (e.g., an element-wise transformation), and the intermediate result is written back to global memory. This result is then immediately read again to serve as the input for the subsequent operation, repeating the cycle. This pattern generates excessive and redundant data movement, saturating the memory bus and leaving the computational cores underutilized.

For instance, consider the computation of $\sin^2(x) + \cos^2(x)$. A framework executing this expression naively would launch multiple separate CUDA kernels: one for $\sin(x)$, another for $\cos(x)$, followed by kernels for the squaring operations and finally the addition. Each kernel requires loading its inputs from and storing its outputs to global memory, creating a substantial I/O overhead for the entire sequence.

Operator fusion addresses this inefficiency by combining multiple operations into a single, consolidated kernel. Instead of transferring intermediate results through global memory, all computations are performed on-chip, with data flowing directly between operations within the high-bandwidth shared memory or registers. In the $\sin^2(x) + \cos^2(x)$ example, a fused kernel would load the input x once, compute the entire expression within a single thread or thread block, and only write the final result back to global memory. This approach dramatically reduces the total memory traffic, shifting the performance bottleneck from memory bandwidth to computational throughput.

This optimization can be achieved by manually writing custom CUDA kernels or, more practically, by leveraging modern compilation tools and frameworks—such as `torch.compile` in PyTorch or XLA (Accelerated Linear Algebra)—that automatically identify and fuse compatible sequences of operations. By minimizing superfluous data movement, operator fusion is a powerful method for maximizing the utilization of available GPU hardware.

4.3 Recomputation

Recomputation is a memory-aware optimization that trades computation for reduced memory bandwidth usage. It addresses the memory bottleneck inherent in the backpropagation algorithm, where intermediate activations computed during the forward pass are traditionally stored to global memory and later retrieved during gradient computation. This pattern generates substantial memory I/O traffic that can limit training performance.

The core idea is to avoid storing intermediate activations and instead recompute them during backpropagation when needed. Consider a chain of three sigmoid operations: out $= \sigma(\sigma(\sigma(x)))$.

Naive implementation

- *Forward pass*: 1 read (input x) + 3 writes (intermediate results $S_1 = \sigma(x)$, $S_2 = \sigma(S_1)$, and final output out $= \sigma(S_2)$)
- *Backward pass*: 3 reads (S_1, S_2, out) + 1 write (dx)
- *Total*: 8 memory operations (4 reads + 4 writes)

Recomputation implementation

- *Forward pass*: 1 read (x) + 1 write (out)
- *Backward pass*: 2 reads (x, $d$out) + 1 write (dx)

- *Total*: 5 memory operations (3 reads + 2 writes)

This strategy reduces total memory access from 8 to 5 operations—a 37.5% reduction—at the cost of recomputing the intermediate sigmoid functions. The trade-off is particularly advantageous when the GPU is memory-bandwidth bound but has available computational capacity. Modern deep learning frameworks such as PyTorch and TensorFlow implement this technique selectively through activation checkpointing, applying it to network segments where the memory savings justify the computational overhead.

4.4 Tiling

Tiling is a fundamental optimization technique designed to minimize global memory accesses by grouping memory operations and leveraging faster on-chip memory. This approach is particularly effective for compute-intensive operations such as matrix multiplication.

In a naive matrix multiplication implementation computing $P = M \times N$, each element of the input matrices must be accessed multiple times from global memory. For instance, element $M(0, 0)$ is read by every thread that computes a value in the first row of P, resulting in redundant and uncoalesced memory accesses. This access pattern significantly degrades performance due to high latency and inefficient bandwidth utilization.

The tiling strategy addresses this by partitioning the input matrices into smaller submatrices (tiles) that fit entirely within the high-bandwidth shared memory. The computation proceeds in phases:

- Load tile $M_{i,j}$ and tile $N_{j,k}$ from global memory into shared memory
- Perform all possible multiply-accumulate operations involving these tiles
- Store the partial results in registers or shared memory
- Repeat until all tiles are processed

This approach reduces global memory traffic by a factor of T, as each tile is loaded from global memory only once but reused for all relevant computations within the shared memory. For an $N \times N$ matrix multiplication, while the naive approach requires each input element to be read N times from global memory, the tiled approach reduces global memory reads to N/T per element by reusing each tile T times.

However, tiling introduces implementation challenges when matrix dimensions are not integer multiples of the tile size, potentially leading to load imbalance and underutilization of streaming multiprocessors (SMs). Optimal tile size selection must therefore account for shared memory capacity, memory access coalescing, load balancing across SMs, and dimension divisibility.

Modern GPU programming frameworks such as CUDA provide abstractions for efficient tiling implementation, while compilers increasingly automate tile size selection based on hardware characteristics and problem dimensions. When properly configured, tiling can transform memory-bound operations into compute-bound ones, significantly improving hardware utilization and overall performance.

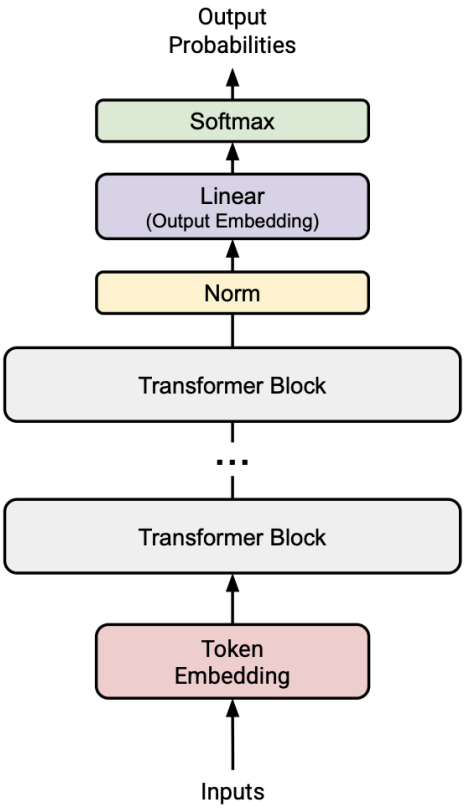

Figure 2: An overview of our Transformer language model.

5 Experiment

5.1 Transformer Implementation

In this work, we implement a decoder-only Transformer language model from scratch. The model, whose high-level architecture is depicted in Figure 2, follows the pre-norm architecture, which has been widely adopted in modern large language models due to its improved training stability. The model consists of token embeddings, multiple Transformer blocks, and output normalization and embedding layers.

Model Structure The token embedding layer maps discrete token IDs to continuous vector representations of dimension d_{model}. These embeddings are then processed by a stack of N identical Transformer blocks. The detailed design of each block is shown in Figure 3. Each block includes a multi-head self-attention mechanism with rotary positional embeddings (RoPE) and a gated feed-forward network using the SwitchLU activation function. Both sub-layers are wrapped with RMSNorm and residual connections, ensuring stable gradient flow during training.

The self-attention mechanism employs a causal mask to prevent the model from attending to future tokens, making it suitable for autoregressive language modeling. Rotary positional embeddings are applied to queries and keys to incorporate relative position information explicitly. The feed-forward network uses a SiLU-based gating mechanism to enhance representational capacity.

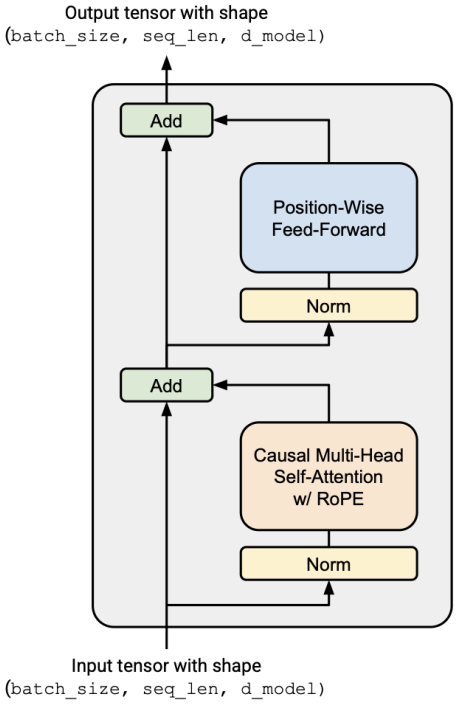

Figure 3: A pre-norm Transformer block.

After processing through all Transformer blocks, a final RMSNorm layer is applied, followed by a linear transformation to produce logits over the vocabulary. The model is implemented efficiently using Einstein summation notation where appropriate, and all parameters are initialized with truncated normal distributions to promote stable training.

Story Generation After building the model architecture, we conducted a small-scale experiment on story generation. The model was trained on the TinyStories dataset, which was preprocessed using custom-built BPE tokenizer. The training procedure employed the cross-entropy loss function, AdamW optimizer, cosine annealing learning rate scheduling, and gradient clipping to ensure stable optimization.

Upon completion of training, we performed text generation through autoregressive inference. Given the prompt *"Once upon a time"*, the model generated the following story:

there was a little boy named Ben. Ben loved to explore the world around him. He saw many amazing things, like beautiful vases that were on display in a store. One day, Ben was walking through the store when he came across a very special vase. When Ben saw it he was amazed! He said, "Wow, that is a really amazing vase! Can I buy it?" The shopkeeper smiled and said, "Of course you can. You can take it home and show all your friends how amazing it is!" So Ben took the vase home and he was so proud of it! He called his friends over and showed them the amazing vase. All his friends thought the vase was beautiful and couldn't believe how lucky Ben was. And that's how Ben found an amazing vase in the store!

The generated story demonstrates the model's ability to maintain narrative coherence, character consistency, and logical event progression, validating the effectiveness of our Transformer implementation for language modeling tasks.

5.2 Flash Attention 2 Triton kernel

The objective of this experiment was to implement the forward and backward passes of the FlashAttention-2 algorithm using Triton, a high-level Python-like language and just-in-time compiler for writing efficient GPU kernels. The performance of this custom Triton implementation was then benchmarked against a baseline PyTorch implementation of standard scaled dot-product attention across a range of sequence lengths and model dimensions.

Implementation The implementation consisted of two main parts:

- **Triton Kernel (`flash_fwd_kernel`):** A JIT-compiled Triton kernel implementing the online softmax tiling strategy central to FlashAttention. The kernel is designed to operate on tiles of queries and keys, both set to 16. For each query tile, it iterates over all key/value tiles, accumulating the output using the stable online softmax calculation . The kernel supports causal masking by applying a negative infinity mask to future positions. It utilizes Triton's `make_block_ptr` and tensor operations like `tl.dot` for efficient memory access and computation.
- **Autograd Function (`TritonFlashAttention2`):** A PyTorch `autograd.Function` that wraps the Triton kernel, handling the device placement, memory layout checks, and kernel launch configuration. The backward pass recomputes the attention matrix S and the probability matrix P from the saved inputs and the output log-sum-exp L to calculate the gradients for Q, K, and V.

Experimental Setup Performance was evaluated using a robust benchmarking script. The key aspects of the setup were:

- **Inputs:** Randomly generated tensors for Q, K, V, and the upstream gradient dO.
- **Configurations:** Tested sequence lengths from 128 to 16384 and head dimensions of 64 and 128, using FP16 precision.
- **Benchmarking Procedure:** For each configuration, both the forward pass and the combined forward+backward pass were timed. The timing function included warm-up runs, multiple measurement repetitions, and trimmed mean calculation to reduce variance and outliers. GPU memory was cleared between configurations.
- **Baseline:** A standard PyTorch implementation of scaled dot-product attention with a causal mask, serving as the performance baseline.

Results and Analysis The benchmarking results, presented in a Figure 4, compare the execution time of the baseline PyTorch attention and the custom Triton FlashAttention-2 implementation across different sequence lengths and model dimensions.

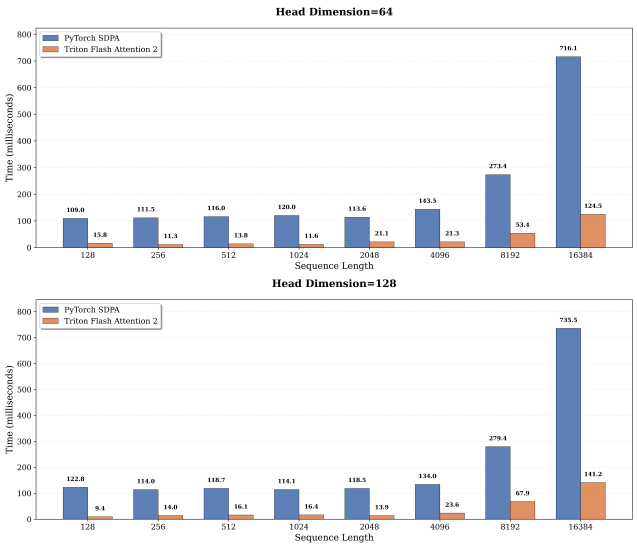

Figure 4: Enter Caption

The Triton FlashAttention-2 implementation demonstrates overwhelming performance superiority, particularly in the backward pass where it achieves 6–12x speedups across all sequence lengths. This advantage is most pronounced at shorter sequences (128–1024), where Triton reduces total computation time from approximately 115ms to just 15ms—representing a 7–9x overall improvement. While PyTorch maintains competitive forward pass latency below seq_len=2048, its performance degrades catastrophically beyond this point, with forward pass latency increasing 100-fold from 0.3ms to 40ms and total time reaching 735ms at seq_len=16384. In contrast, Triton maintains reasonable scaling characteristics, reaching only 141ms total time at the maximum sequence length—a 5x advantage over PyTorch.

The benchmarking results confirm the significant performance improvements offered by the FlashAttention algorithm over the standard implementation for longer sequences, validating its importance in training and inferring with large language models on modern GPU hardware. The Triton implementation effectively reduces the memory bottleneck associated with the attention mechanism.

5.3 Profiling and Benchmarking

To comprehensively evaluate the performance characteristics of hardware-efficient attention mechanisms, we conducted extensive profiling and benchmarking experiments comparing standard PyTorch SDPA, SageAttention, and Sparse SageAttention across various sequence lengths and attention configurations.

Experimental Setup The benchmarking experiments were performed on an NVIDIA A800 80GB PCIe GPU with the following software environment: Python 3.13.7, PyTorch 2.8.0+cu128, CUDA 12.8, and cuDNN 91002. We evaluated each attention mechanism using a batch size of 4 and 32 attention heads, testing sequence lengths from 1K to 8K tokens. Both causal and non-causal attention modes were examined, with a focus on head dimension 64. We employed two key metrics for comprehensive evaluation:

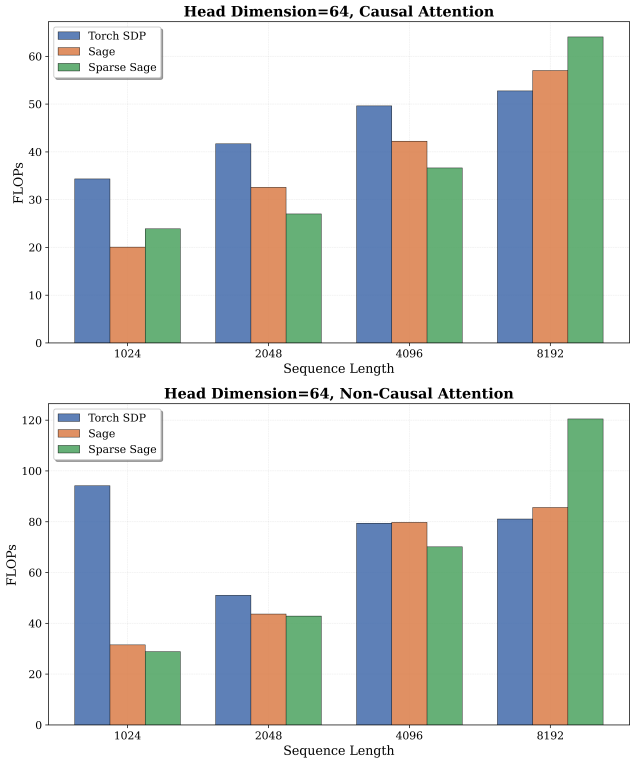

Figure 5: TFLOPS comparison of different attention mechanisms

- **Computational Throughput**: Measured in TFLOPS (Tera Floating Point Operations Per Second) to assess hardware utilization efficiency
- **Precision Accuracy**: Cosine similarity between each method's output and the reference PyTorch SDPA implementation

Results and Analysis The benchmarking results reveal several key insights:

- **Performance Scaling**: As shown in Figure 5, both SageAttention variants demonstrate superior computational efficiency compared to standard SDPA at longer sequence lengths. While SDPA achieves higher TFLOPS for shorter sequences (1K-4K), SageAttention and Sparse SageAttention show better scaling characteristics, with Sparse SageAttention reaching 64.05 TFLOPS at 8K sequence length in causal mode, outperforming SDPA's 52.75 TFLOPS.
- **Precision-Runtime Tradeoff**: The hardware-efficient methods maintain excellent precision preservation with cosine similarity consistently above 0.989 across all configurations. The observed L1 errors remain below 0.044, indicating acceptable numerical deviation for most practical applications.

- **Causal vs. Non-Causal Patterns**: The performance characteristics differ significantly between causal and non-causal attention. Non-causal attention generally achieves higher TFLOPS due to the absence of masking constraints, but shows slightly reduced precision compared to causal attention.

These results confirm that hardware-efficient attention methods provide compelling tradeoffs, offering substantially improved memory efficiency and better scaling to long sequences while maintaining acceptable precision levels for practical deployment.

5.4 Innovations on Sage3

Experimental Setup The evaluation utilized the Llama-3.1-8B-4K model to assess the performance and precision of FP4 quantized attention mechanisms. The experimental setup included the following key components:

- **Model**: Llama-3.1-8B-4K with Q, K, and V tensors
- **Quantization**: Standard E2M1 quantization
- **Metrics**: Cosine similarity, L1 loss, and RMSE
- **Baseline**: Full-precision scaled dot-product attention from PyTorch

Problem Identification During the experimental evaluation of the Sage3 attention mechanism using FP4 quantization on the Llama-3.1-8B-4K model (specifically its 1-qkv layer), a significant precision degradation was observed. The initial results showed a cosine similarity of 0.571400, an L1 loss of 1.743900, and an RMSE of 0.011500. Under ideal quantization conditions, the expected metrics should maintain a Cossim above 0.995 and an L1 loss below 0.08, indicating that the observed error was substantial and unacceptable for practical deployment.

Cause Analysis To diagnose the source of the error, an ablation study was conducted by selectively applying quantization to the query, key, and value matrices. The results are summarized in Table 4.

Table 4: Precision metrics under different quantization settings.

Configuration	Cossim	L1	RMSE
Quantize Q, K, V	0.5714	1.7439	0.0115
No Quantization	0.9964	0.0862	0.0007
Quantize K, V only	0.5806	1.7330	0.0114
Quantize Q, V only	0.5811	1.7289	0.0114
Quantize Q, K only	0.9731	0.2038	0.0020

The results clearly indicate that quantizing the value matrix contributed most significantly to the precision loss. Further analysis of the quantization error per matrix revealed the following:

A detailed inspection of a representative block (first batch, first head, first row of 16 values) from the V matrix uncovered the core issue: the scaling factor, `scale_x`, was being truncated to zero during the conversion `scale_x.to(torch.float8_e4m3fn).to(torch.float32)`. This occurred because the maximum value in the block was exceedingly small (0.00154781), leading to a scale factor (0.00025797) that underflowed in FP8 representation. Consequently, all the dequantized values were zero, destroying the information in the value matrix.

Table 5: Quantization error per matrix.

Matrix	Cossim	L1	RMSE
Q	0.9953	0.0953	0.0681
K	0.9953	0.0932	0.1478
V	0.9431	0.3432	0.0018

Proposed Solution Inspired by the two-stage quantization method used for the P matrix in the original Sage3 design(Zhang et al. 2025a) , we implemented a similar strategy for the V matrix. This approach uses a higher-precision intermediate representation to compute the scale factor, preventing underflow.

The revised quantization process for the same block of values yielded drastically improved results:

- The scale factor was correctly preserved as `32.0` instead of being truncated to `0`.
- The dequantized values closely approximated the original tensor.

The final evaluation confirmed the effectiveness of the fix: the cosine similarity with the full-precision baseline recovered to 0.9561, the L1 loss was reduced to 0.3216, and the RMSE dropped to 0.0025. This represents a major improvement over the initial implementation and demonstrates the necessity of careful scaling factor handling for ultra-low precision quantization on activations with small magnitudes.

6 Conclusion

This paper focused on hardware-efficient attention methods, which accelerate transformer attention through hardware-aware algorithms without changing its mathematical essence. We reviewed key approaches including FlashAttention, SageAttention, and SpargeAttention, and discussed their underlying GPU acceleration techniques such as operator fusion, tiling, and low-precision computation.

Through experiments including a custom Triton kernel implementation and innovations in Sage3, we demonstrated that these methods achieve significant improvements in both speed and memory usage while maintaining accuracy—especially critical for long-sequence applications.

Future work may explore adaptive attention mechanisms, stronger hardware-algorithm co-design, and optimizations for emerging accelerators. This study offers a practical overview and validation of hardware-efficient attention, providing valuable insights for developing high-performance transformer models.

References

Dao, T. 2023. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*.

Dao, T.; Fu, D.; Ermon, S.; Rudra, A.; and Ré, C. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35: 16344–16359.

Shah, J.; Bikshandi, G.; Zhang, Y.; Thakkar, V.; Ramani, P.; and Dao, T. 2024. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. *Advances in Neural Information Processing Systems*, 37: 68658–68685.

Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, Ł.; and Polosukhin, I. 2017. Attention is all you need. *Advances in neural information processing systems*, 30.

Zhang, J.; Huang, H.; Zhang, P.; Wei, J.; Zhu, J.; and Chen, J. 2024a. Sageattention2: Efficient attention with thorough outlier smoothing and per-thread int4 quantization. *arXiv preprint arXiv:2411.10958*.

Zhang, J.; Wei, J.; Huang, H.; Zhang, P.; Zhu, J.; and Chen, J. 2024b. Sageattention: Accurate 8-bit attention for plug-and-play inference acceleration. *arXiv preprint arXiv:2410.02367*.

Zhang, J.; Wei, J.; Zhang, P.; Xu, X.; Huang, H.; Wang, H.; Jiang, K.; Zhu, J.; and Chen, J. 2025a. Sageattention3: Microscaling fp4 attention for inference and an exploration of 8-bit training. *arXiv preprint arXiv:2505.11594*.

Zhang, J.; Xiang, C.; Huang, H.; Xi, H.; Zhu, J.; Chen, J.; et al. 2025b. SpargeAttention: Accurate and Training-free Sparse Attention Accelerating Any Model Inference. In *Forty-second International Conference on Machine Learning*.

Zhang, J.; Xu, X.; Wei, J.; Huang, H.; Zhang, P.; Xiang, C.; Zhu, J.; and Chen, J. 2025c. Sageattention2++: A more efficient implementation of sageattention2. *arXiv preprint arXiv:2505.21136*.